INDIAN INSTITUTE OF TECHNOLOGY, PATNA

MA3206:

AI Assignment

Report – Lab 1 Assignment

Submitted To:
Dr. Ranjeet Ranjan Jha

Submitted By:
Aditya Gupta
2301MC51

Aditya Onam
2301MC60

Shakib Akhtar
2301MC49

Task 1

Topic: Binary Classification (Breast Cancer FNA)

1. Objective

The objective of this task was to implement the K-Nearest Neighbors (KNN) algorithm from scratch to classify breast mass samples as either Malignant (M) or Benign (B). The model was trained on the Wisconsin Breast Cancer Diagnostic dataset, utilizing 30 continuous features such as radius, texture, and smoothness.

2. Methodology

- **Data Preprocessing:**
  - **Normalization:** All feature values were normalized using Min-Max scaling to ensure that features with larger magnitudes (e.g., Area) did not dominate the distance calculations.
  - **Splitting:** The data was divided into 80% training (456 samples) and 20% testing (113 samples).
- **Algorithm:**
  - A custom KNN class was implemented without relying on libraries like sklearn.
  - Five distance metrics were implemented manually: *Euclidean, Manhattan, Minkowski (p=3), Cosine, and Hamming*.
- **Experimentation:**
  - The model was tested across varying K values: [3, 4, 9, 20, 47].

3. Experimental Results

3.1 Best Model Configuration

After running experiments across all combinations of K and distance metrics, the optimal configuration was identified:

- **Best K:** 9
- **Best Distance Metric:** Manhattan Distance (L1 Norm)

3.2 Performance Metrics

The performance of the best model (K=9, Manhattan) on the test data is summarized below:

Metric	Value	Description
Accuracy	**97.35%**	Overall correctness of the model.
Precision	**97.56%**	Out of all predicted Malignant cases, 97.56% were actually Malignant.
Recall	**95.24%**	The model successfully identified 95.24% of all actual Malignant cases.

Confusion Matrix:

- **True Negatives (70):** Benign cases correctly identified as Benign.
- **False Positives (1):** One Benign case was incorrectly flagged as Malignant.
- **False Negatives (2):** Two Malignant cases were missed.
- **True Positives (40):** Malignant cases correctly identified.

3.3 Visual Analysis

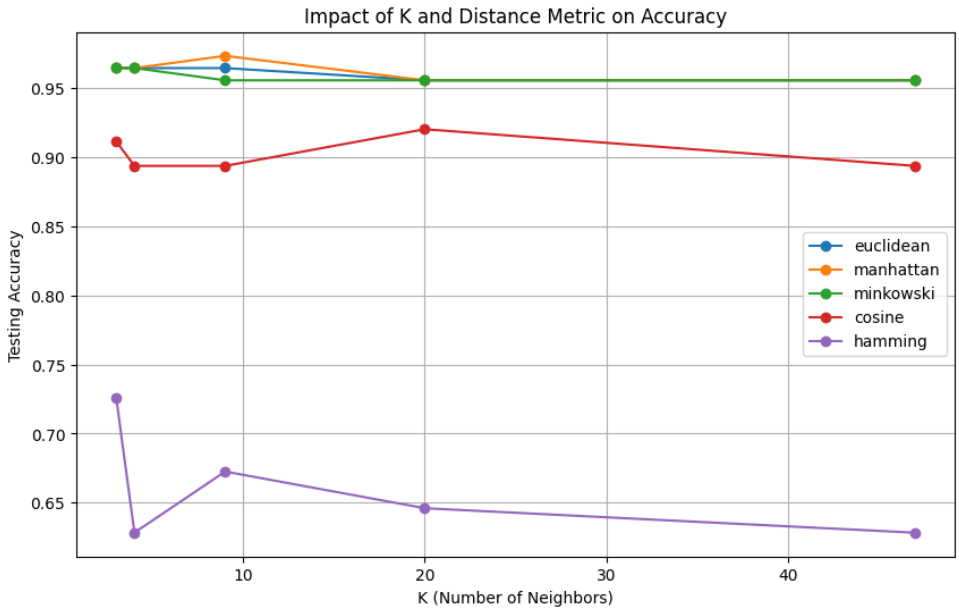

Figure 1: Impact of K and Distance Metric on Accuracy

The plot illustrates the testing accuracy for different K values across five distance metrics.

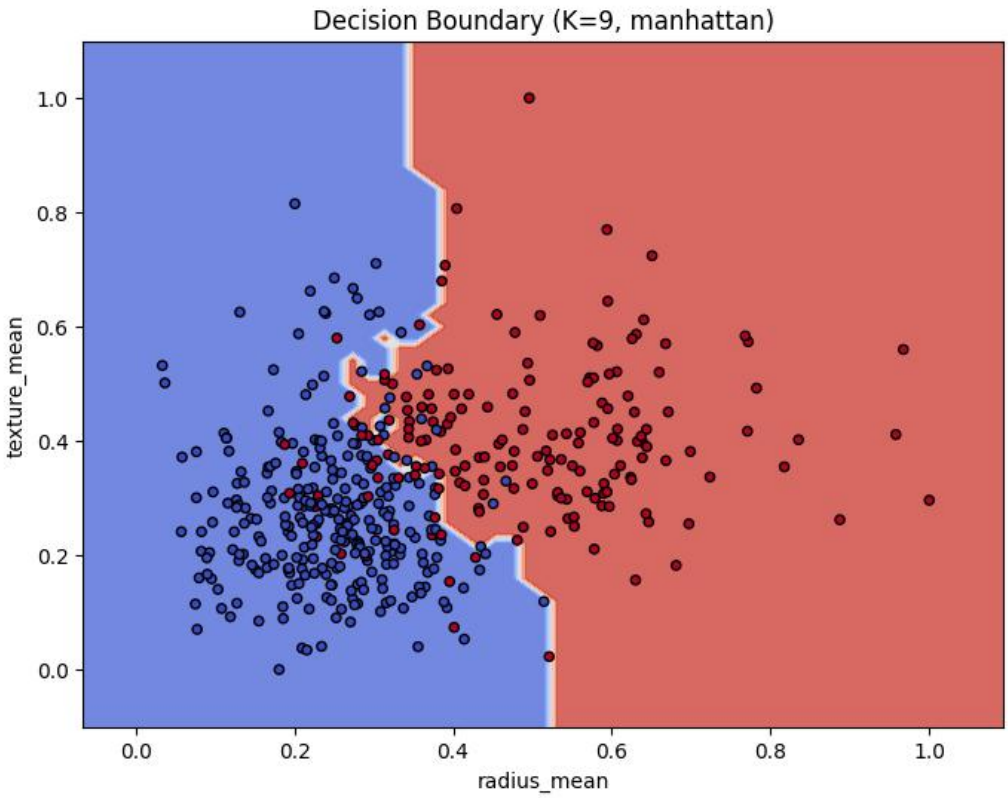

Figure 2: Decision Boundary Visualization

The decision boundary for the best model (K=9, Manhattan) visualized using the first two features (Radius Mean vs. Texture Mean).

4. Inferences and Observations

Based on the experimental results and the plots above, the following inferences were drawn:

1. **Failure of Hamming Distance:**
    - As observed in *Figure 1* (the purple line), **Hamming distance performed significantly worse** than all other metrics, with accuracy hovering around 60-70%.
    - **Reason:** Hamming distance calculates the number of positions at which corresponding symbols are different. Since our dataset consists of **continuous floating-point numbers** (e.g., 0.123 vs 0.124), almost every pair of points is considered "different" in every dimension. Hamming distance is suited for categorical/binary data, not continuous medical measurements.

2. **Superiority of Manhattan Distance:**
    - **Manhattan (L1 norm)** achieved the highest accuracy (97.35%), slightly edging out Euclidean (L2).
    - **Reason:** In higher-dimensional spaces (30 features), the contrast between the nearest and farthest neighbors can diminish (Curse of Dimensionality). L1 norm (Manhattan) is often more robust and distinguishes distances better in high dimensions compared to L2 (Euclidean), which is sensitive to outliers due to the squaring of differences.
3. **Impact of K Value:**
    - **K=9 (Optimal):** Provided the best balance between bias and variance. The decision boundary (*Figure 2*) is smooth enough to ignore noise but detailed enough to capture the class separation.
    - **Small K (3, 4):** Slightly lower accuracy likely due to overfitting (sensitivity to local noise).
    - **Large K (47):** Accuracy remained high but generally plateaus. Excessive K can lead to underfitting, where the boundary becomes too linear and misses local clusters.

Task 2

Topic: Multi-Class Classification (CIFAR-10 Dataset)

1. Objective

The objective of this task was to build a K-Nearest Neighbors (KNN) classifier from scratch to categorize images from the CIFAR-10 dataset into 10 distinct classes (e.g., Airplane, Car, Bird, Cat). Given the high dimensionality of image data (3072 features per image) and the large dataset size (60,000 images), a GPU-accelerated approach using PyTorch was implemented to ensure efficiency.

2. Methodology

- **Data Processing:**
  - **Flattening:** Each 32 x 32 color image was flattened into a single vector of size 3072 (32 x 32 x 3).
  - **Normalization:** Pixel values were scaled from [0, 255] to [0.0, 1.0].
  - **Scale:** The model utilized the **full dataset** (50,000 training images, 10,000 test images) thanks to GPU acceleration.

- **Algorithm & Optimization:**
  - **PyTorch & GPU:** The custom KNN model utilized PyTorch tensors on a CUDA-enabled GPU (T4).
  - **Vectorized Distances:** Loops were replaced with matrix operations (e.g., torch.cdist, matrix multiplication) to compute distances for entire batches of test images simultaneously.

- **Experimentation:**
  - **K Values:** Tested [3, 4, 9, 20].
  - **Metrics:** Euclidean, Manhattan, and Cosine Similarity.

3. Experimental Results

3.1 Best Model Configuration

Experiments revealed that the "Manhattan" distance metric consistently outperformed Euclidean and Cosine similarity for this image dataset.

- **Best K:** 9
- **Best Distance Metric:** Manhattan Distance (L1 Norm)

3.2 Performance Metrics

The performance of the best configuration (K=9, Manhattan) on the full 10,000 image test set is as follows:

Metric	Value	Description
Accuracy	**38.64%**	Correctly classified 3,864 out of 10,000 images.
Precision	**46.43%**	Macro-average precision across all 10 classes.
Recall	**38.64%**	Macro-average recall across all 10 classes.

Note: While 38.64% appears low compared to modern Deep Learning standards (95%+), it is a typical and expected result for a raw-pixel KNN classifier, which lacks feature extraction capabilities.

3.3 Visual Analysis

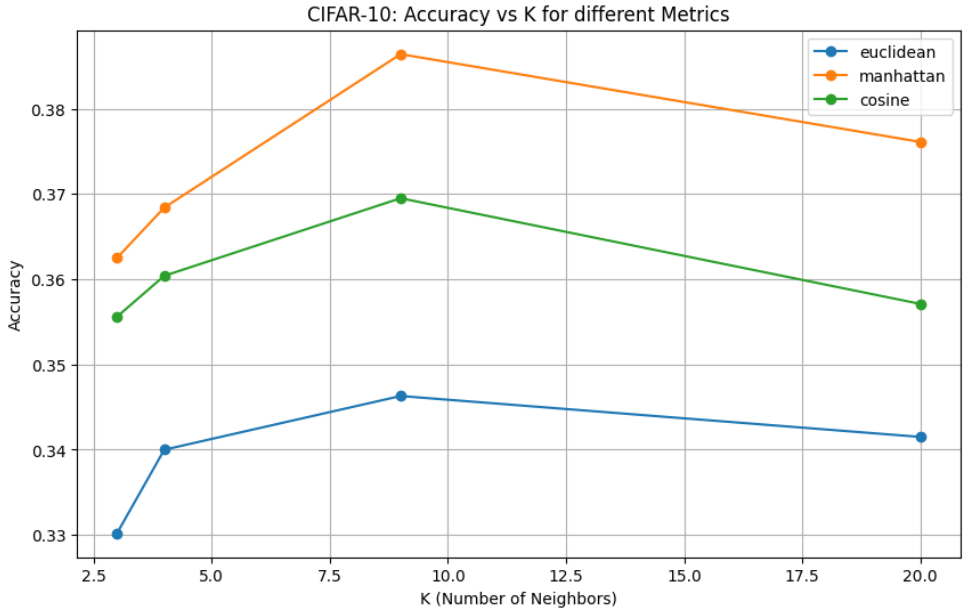

Figure 1: Accuracy vs. K for Different Metrics

- **Observation:** The orange line (Manhattan) is strictly superior to Green (Cosine) and Blue (Euclidean) at every K value.
- **Trend:** Accuracy peaks at **K=9** for all metrics and begins to degrade or plateau at K=20.

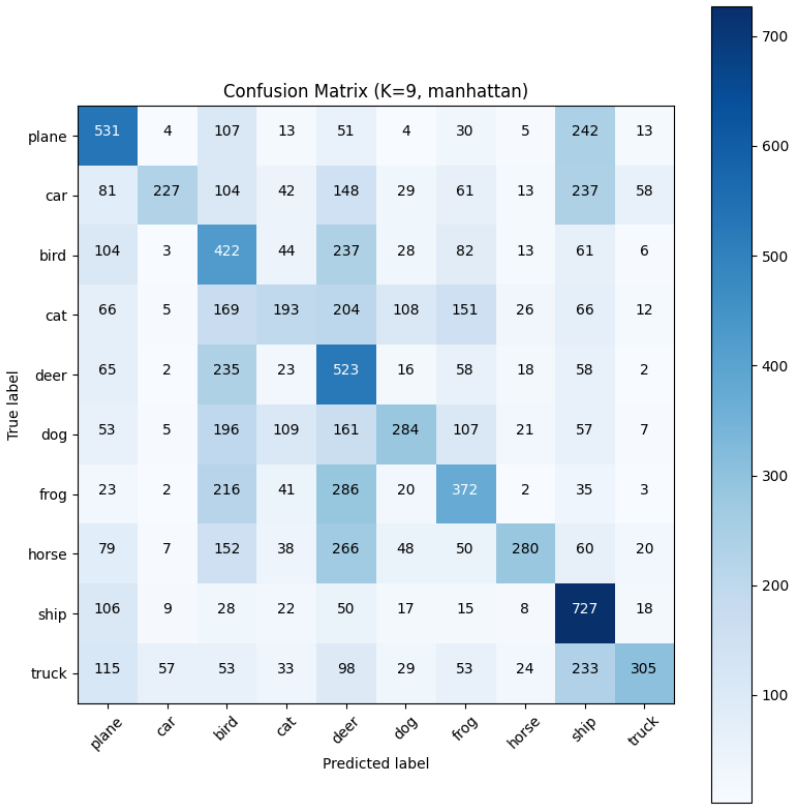

Figure 2: Confusion Matrix (K=9, Manhattan)

- **Strongest Class: Ship** (Index 8) had the highest correct predictions (727).
- **Weakest Class: Cat** (Index 3) was highly confused, with only 193 correct predictions. It was frequently misclassified as **Deer** (204) or **Frog** (151).
- **Common Confusions:**
    - **Planes vs. Ships:** 242 Planes were misclassified as Ships. This is likely due to the similarity in background color (blue sky vs. blue water).
    - **Animals vs. Animals:** There is significant confusion within the animal classes (Bird, Cat, Deer, Dog, Frog, Horse) due to similar textures and lack of distinct rigid shapes compared to vehicles.

4. Inferences and Observations

1. **Superiority of Manhattan Distance (L1):**
    - Manhattan distance achieved ~38.6% accuracy compared to Euclidean's ~34.6%.
    - **Reason:** In high-dimensional spaces (3072 dimensions), Euclidean distance (L2) is sensitive to outliers (large pixel differences). Manhattan distance (L1) sums the absolute differences, making it more robust to noise in pixel intensity (e.g., a bright spot in the background).
2. **GPU Acceleration & Efficiency:**
    - **Cosine Similarity** was the fastest (~1.4s), followed by Euclidean (~2s). Manhattan was significantly slower (~26s) on the GPU.
    - **Reason:** Euclidean and Cosine calculations can be reduced to highly optimized Matrix Multiplications (Dot Products). Manhattan distance requires element-wise subtraction and summation, which is computationally heavier for the GPU to batch process effectively compared to dot products.
3. **Limitations of Raw Pixel KNN:**
    - The model relies on pixel-to-pixel overlap. If a "Cat" is shifted 5 pixels to the right, the KNN sees it as a completely different vector. This explains the confusion between classes with similar color palettes (Plane/Ship) rather than similar shapes.